UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Sistemas Operativos I

Simulador de Funciones del Kernel de un Sistema Operativo

Reporte técnico individual con estructura APA

Autora: Marli Xiomara Vásquez Pérez

Carné y sección: no proporcionados

Docente: Ing. Julio Gomez

3 de noviembre de 2026

Repositorio: https://github.com/mvasquezp31-create/Simulador_kernel

Resumen

Se desarrolla una simulación discreta de un kernel con interfaz de consola.
El modelo integra PCB, planificación Round Robin, memoria contigua de 100 MB,
semáforo binario y dos recursos exclusivos. Se observa el comportamiento de
cada módulo mediante comandos y bitácoras. La implementación es Java 17+
sin dependencias externas. No ejecuta procesos ni administra hardware real.
Se incorporan pruebas reproducibles y guiones para una demostración de
hasta 15 minutos. Palabras clave: kernel, procesos, RAM, sincronización.

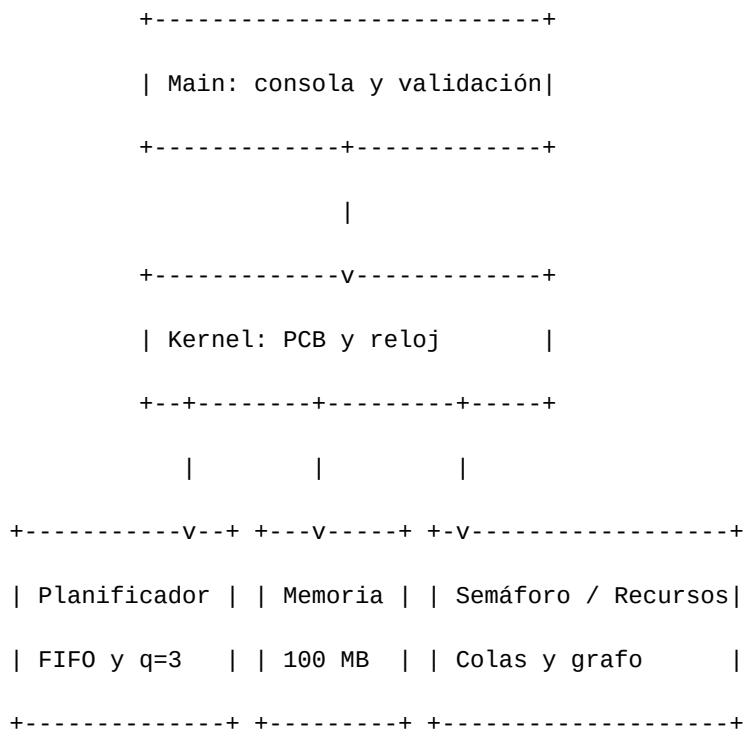
1. Introducción y objetivos

El proyecto permite visualizar cómo un sistema operativo comparte CPU,
admite procesos en memoria y coordina accesos a datos y recursos.
Su objetivo general es modelar las funciones solicitadas en el enunciado.
Objetivos específicos: representar PCB y estados; aplicar quantum fijo 3;
validar admisión en 100 MB; proteger una variable compartida; provocar y

resolver espera circular; y aceptar parámetros aleatorios de forma robusta.

2. Arquitectura del simulador

El núcleo coordina módulos con responsabilidades independientes.



Proceso almacena PID, estado, memoria, ráfagas total/restante, llegada, primera ejecución, finalización, espera en listos y tiempo bloqueado. Planificador conserva una cola FIFO sin PID duplicados. Memoria mantiene bloques ordenados por dirección. SemaforoBinario tiene propietario y cola propia. GestorRecursos registra los propietarios de R1/R2 y sus solicitantes. Kernel realiza cambios coherentes al bloquear, despertar, terminar y abortar. Los PCB terminados se conservan para inspección; su memoria queda liberada.

Supuestos: un procesador, CPU en unidades abstractas y costo de contexto

cero. Solo ejecutar CPU avanza el reloj; los comandos son eventos atómicos simulados entre despachos. No se modelan E/S, prioridades ni hardware.
La terminación y el aborto liberan RAM, semáforo y recursos y despiertan solicitantes. La simulación no persiste al cerrar el programa.
El documento anuncia cinco componentes, pero solo enumera A, B, C y D: se implementan los cuatro apartados descritos sin inventar un requisito.

3. Análisis teórico

3.1. Round Robin

Round Robin limita cada turno a un quantum e intercala los procesos listos. Esta política permite atender otros trabajos antes de que finalice toda la ráfaga del primero (Arpaci-Dusseau y Arpaci-Dusseau, s. f., Scheduling).
Aquí consumo=min(3, CPU restante). Si resta CPU se guarda contexto y el PID regresa al final de la cola; si termina libera sus reservas.

Ejemplo comprobado: llegadas en 0; ráfagas A=5, B=4, C=2.
Gantt: [0,3) A; [3,6) B; [6,8) C; [8,10) A; [10,11) B.
Retorno=finalización-llegada.
Respuesta=primera ejecución-llegada.
Espera en listos=retorno-CPU consumida-tiempo bloqueado.
En procesos terminados sin bloqueos, espera=retorno-ráfaga total.

Proceso	Llegada	CPU	Finaliza	Retorno	Espera	Respuesta
A	0	5	10	10	5	0
B	0	4	11	11	7	3
C	0	2	8	8	6	6

Espera promedio=(5+7+6)/3=6 unidades.

Retorno promedio=(10+11+8)/3=9.6667 unidades.

Respuesta promedio=(0+3+6)/3=3 unidades.

La bitácora registra cada expiración de quantum; despachos cuenta cada turno. Cambios entre PID cuenta solo despachos consecutivos de procesos distintos. Un quantum menor implicaría más sobrecarga en hardware real, costo que esta simulación declara explícitamente como cero.

3.2. Gestión de memoria

Se eligió asignación contigua dinámica first-fit. Se recorre la RAM simulada en [0,100) para encontrar el primer hueco suficiente. Reserva y PCB se crean solo si la admisión tiene éxito. Un rechazo Out of Memory no consume PID. Liberar un bloque permite reutilizarlo. No existe compactación automática: 40 MB libres en huecos de 30 y 10 no admiten un bloque contiguo de 35 MB. El recorrido de n bloques cuesta O(n); el árbol mantiene las direcciones ordenadas. Toda memoria se expresa en MB enteros, con valores positivos.

3.3. Concurrencia y sincronización

Un semáforo binario parte disponible (1). wait(pid) modela wait(): obtiene permiso o bloquea en una cola específica; signal(pid) modela signal(): transfiere permiso en orden FIFO o deja el semáforo libre.

Estas primitivas modelan el control de acceso a una sección crítica (Arpaci-Dusseau y Arpaci-Dusseau, s. f., Semaphores).

El simulador añade propietario para rechazar liberación ajena y adquisición recursiva. B bloqueado no está en listos ni puede modificar la variable.

La prueba sin protección intercala dos lecturas de 0 y dos escrituras de 1: se obtiene 1 en lugar de 2. Con el semáforo implementado, A escribe primero, transfiere a B y el resultado es 2. Es concurrencia lógica reproducible, no

una carrera ejecutada con hilos reales del equipo.

3.4. Interbloqueo

Cada recurso admite un propietario. Si está ocupado, el solicitante retiene sus otros recursos y espera. Una arista del solicitante al propietario representa la dependencia. Repetir un PID al seguir aristas indica un ciclo.
A posee R1 y pide R2; B posee R2 y pide R1: A -> B -> A.
Se cumplen exclusión mutua, retención y espera, ausencia de expropiación ordinaria y espera circular (Arpaci-Dusseau y Arpaci-Dusseau, s. f., Common Concurrency Problems). Abortar una víctima es una recuperación explícita: libera RAM y recursos, retira solicitudes y despierta al otro PID.
El detector cubre R1/R2, no ciclos mixtos con el semáforo de sección crítica.

4. Bitácora de casos de prueba

Los anexos son volcados de texto de ejecuciones reales, modalidad admitida por el enunciado. Verificar.ps1 compila, ejecuta las pruebas, recorre los mismos comandos de la CLI y regenera este PDF con sus resultados.
Caso RAM: admitir 60 MB, rechazar 41, abortar y admitir 100 MB.
Caso RR: verificar A-B-C-A-B y tiempos de espera 5,7,6.
Caso semáforo: B espera, su escritura se rechaza y tras signal el total es 2.
Caso deadlock: detectar A-B-A, abortar A y permitir que B termine.
Caso estrés: 500 operaciones con semilla 20261103, conservando ocupación entre 0 y 100 y cola sin duplicados. Se prueban además negativos, ceros, números enormes, PID inexistentes, recursos inválidos y comandos mal formados.
Las entradas erróneas producen un mensaje y la CLI continúa funcionando.
Ejecutar tiene un límite configurable de turnos y conserva estado al alcanzarl
o.

5. Conclusiones

El simulador permite observar el intercambio de CPU y la admisión de procesos sin depender de un sistema operativo real. Distinguir colas de listos y de bloqueados evita despachar procesos sin permiso. La liberación coordinada permite recuperar memoria y romper el interbloqueo sin dejar reservas huérfana s.

Las métricas se corresponden con cálculos teóricos bajo los supuestos elegidos .

Las pruebas deterministas y de estrés respaldan las validaciones para la exposición. La presentación individual fue solicitada por la autora, aunque el enunciado original describe una demostración grupal; confirmar esa modalida d

con el docente. Carné y sección quedan pendientes de completar por la autora.

Referencias

Arpaci-Dusseau, R. H., y Arpaci-Dusseau, A. C. (s. f.). Scheduling: Introduction. En Operating Systems: Three Easy Pieces. https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf

Arpaci-Dusseau, R. H., y Arpaci-Dusseau, A. C. (s. f.). Semaphores. En Operating Systems: Three Easy Pieces. https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf

Arpaci-Dusseau, R. H., y Arpaci-Dusseau, A. C. (s. f.). Common Concurrency Problems. En Operating Systems: Three Easy Pieces. https://pages.cs.wisc.edu/~remzi/OSTEP/threads-bugs.pdf

Universidad Mariano Gálvez de Guatemala. (s. f.). Enunciado del Proyecto Final: Sistema Operativo I [Documento de la asignatura proporcionado].

Consulta de fuentes: 3 de noviembre de 2026.

ANEXO: docs/evidencias.txt

CASO 1. Out of Memory y recuperación de RAM

OK Rechazo inmediato con RAM insuficiente -> Out of Memory: se necesitan 41 MB contiguos; libres=40 MB. Puede existir fragmentación externa.

OK OOM no deja PCB ni reserva parcial

OK Abortar devuelve toda la RAM

OK OOM no consume PID

OK Terminar libera RAM

OK Memoria cero -> PID y memoria deben ser positivos.

OK Memoria negativa -> PID y memoria deben ser positivos.

OK Memoria extrema sin desbordamiento -> Out of Memory: se necesitan 2147483647 MB contiguos; libres=100 MB. Puede existir fragmentación externa.

OK Ráfaga cero -> Ráfaga: 1 a 1000000 unidades.

OK Ráfaga fuera de rango -> Ráfaga: 1 a 1000000 unidades.

OK Nombre vacío -> Nombre: 1 a 30 letras, números, _ o -.

OK PID inexistente -> PID inexistente: 999

OK Fragmentación: 40 MB libres pero ningún hueco de 35 -> Out of Memory: se necesitan 35 MB contiguos; libres=40 MB. Puede existir fragmentación externa.

OK Huecos contiguos se reutilizan con first-fit

Reloj=0 q=3 despachos=0 cambios entre PID=0

RAM: 90/100 MB; libres=10 MB; bloques=[Bloque[pid=4, inicio=0, tamanio=60], Bloque[pid=3, inicio=60, tamanio=30]]

Listos: [3, 4]

Semáforo=1 propietario=null espera=[]

Recursos (recurso=PID): {}; solicitudes (PID=recurso): {}

Variable global=0

Deadlock: []

1 F1 ABORTADO RAM= 30 MB CPU=2/2 espera=0 bloqueo=0 respuesta=-1 fin=0

2 F2 ABORTADO RAM= 30 MB CPU=2/2 espera=0 bloqueo=0 respuesta=-1 fin=0

3 F3 LISTO RAM= 30 MB CPU=2/2 espera=0 bloqueo=0 respuesta=-1 fin=-1

4 Reutilizar LISTO RAM= 60 MB CPU=1/1 espera=0 bloqueo=0 respuesta=-1 fin=-1

CASO 2. Round Robin q=3; llegadas simultáneas y costo de contexto cero

CPU [0,3) PID=1 restante=2

CPU [3,6) PID=2 restante=1

CPU [6,8) PID=3 restante=0

CPU [8,10) PID=1 restante=0

CPU [10,11) PID=2 restante=0

OK RR requiere cinco despachos

OK Secuencia exacta A B C A B

OK Tiempos de finalización 10, 11, 8

OK Tiempos de espera 5, 7, 6; promedio 6

OK Tiempos de respuesta 0, 3, 6

OK CPU total y liberación de RAM

OK CPU sin listos no inventa avance temporal

OK Un proceso terminado no vuelve a ejecutarse -> El proceso ya terminó o fue abortado.

OK Límite de turnos inválido -> Turnos: 1 a 100000.

OK Ejecución acotada mantiene estado para continuar

OK Llegadas posteriores usan el reloj vigente

CASO 3. Sección crítica y cola FIFO del semáforo

Intercalado sin protección: leer A=0, leer B=0, escribir A=1, escribir B=1. Re
sultado=1; esperado=2.

OK No se modifica variable global sin permiso -> Ejecute wait PID antes de mod
ificar la variable global.

OK A obtiene wait

OK B y C esperan en cola específica

OK Bloqueados no permanecen en listos

OK B bloqueado no escribe en sección crítica -> El proceso está bloqueado y no
puede efectuar esa operación.

OK Un no propietario no libera semáforo -> Solo el propietario puede ejecutar
signal.

OK signal transfiere a B en orden FIFO

OK Con semáforo se conservan ambas actualizaciones

OK Abortar propietario despierta al siguiente

OK Semáforo vuelve a libre

OK wait no reentrante rechazado sin corromper colas -> El semáforo no es reent
rante.

OK Abortar solicitante lo retira de la cola

OK Desbordamiento de variable global rechazado -> long overflow

OK Suma fallida conserva el valor anterior

Reloj=0 q=3 despachos=0 cambios entre PID=0

RAM: 10/100 MB; libres=90 MB; bloques=[Bloque[pid=1, inicio=0, tamanio=10]]

Listos: [1]

Semáforo=0 propietario=1 espera=[]

Recursos (recurso=PID): {}; solicitudes (PID=recurso): {}

Variable global=9223372036854775807

Deadlock: []

1 A LISTO RAM= 10 MB CPU=12/12 espera=0 bloqueo

=0 respuesta=-1 fin=-1

2 B ABORTADO RAM= 10 MB CPU=12/12 espera=0 bloqueo

=0 respuesta=-1 fin=0

3 C ABORTADO RAM= 10 MB CPU=12/12 espera=0 bloqueo

=0 respuesta=-1 fin=0

CASO 4. Detección y resolución de Deadlock

Reloj=0 q=3 despachos=0 cambios entre PID=0

RAM: 80/100 MB; libres=20 MB; bloques=[Bloque[pid=1, inicio=0, tamanio=40], Bl

oque[pid=2, inicio=40, tamanio=40]]

Listos: []

Semáforo=1 propietario=null espera=[]

Recursos (recurso=PID): {1=1, 2=2}; solicitudes (PID=recurso): {1=2, 2=1}

Variable global=0

Deadlock: [1, 2, 1]

1 A BLOQUEADO_RECURSO RAM= 40 MB CPU=5/5 espera=0 bloqueo=0

respuesta=-1 fin=-1

2 B BLOQUEADO_RECURSO RAM= 40 MB CPU=5/5 espera=0 bloqueo=0

respuesta=-1 fin=-1

OK Grafo detecta ciclo A -> B -> A

OK Deadlock bloquea ambos PCB

OK Planificador alerta sin quedar en bucle

OK Abortar A rompe el ciclo y despierta B

OK B recibe R1; RAM de A liberada

OK El superviviente finaliza y libera todos los recursos

t=0 CREAR PID=1 nombre=A RAM=40 CPU=5

t=0 CREAR PID=2 nombre=B RAM=40 CPU=5

t=0 SOLICITAR PID=1 R1 concedido

t=0 SOLICITAR PID=2 R2 concedido

t=0 SOLICITAR PID=1 R2 BLOQUEADO

t=0 SOLICITAR PID=2 R1 BLOQUEADO

t=0 ALERTA DEADLOCK: [1, 2, 1]

t=0 DEADLOCK [1, 2, 1]; use abortar PID.

t=0 DESPERTAR PID=2

t=0 ABORTADO PID=1: RAM y recursos liberados

t=3 CPU [0,3) PID=2 restante=2

t=3 FIN QUANTUM: guardar contexto PID=2 y enviar al final de listos

t=5 CPU [3,5) PID=2 restante=0

t=5 TERMINADO PID=2: RAM y recursos liberados

OK Deadlock requiere dos PID diferentes -> Seleccione dos PID diferentes y amb

os recursos libres.

OK Fallo de precondición no reserva recursos parciales

OK Recurso inexistente -> El recurso debe ser 1 o 2.

OK No volver a solicitar recurso propio -> El proceso ya posee ese recurso.

OK No liberar recurso ajeno -> El proceso no posee ese recurso.

OK Espera simple no es deadlock

OK Abortar elimina solicitudes pendientes

OK Liberación normal deja recursos libres

OK Consola admite tabulaciones

OK Consola sobrevive a entrada: crear X no 3

OK Consola sobrevive a entrada: crear X 10

OK Consola sobrevive a entrada: crear X 1 9999999999999

OK Consola sobrevive a entrada: inventado

OK Consola sobrevive a entrada: sumar 1 999999999999999999999999

OK Consola sobrevive a entrada: paso basura

OK Consola sobrevive a entrada: wait -2

OK Consola imprime éxito y errores recuperables

OK Salida explícita

OK Estrés reproducible: 500 operaciones mantienen RAM y cola válidas

OK Estrés: todos los procesos admitidos finalizan

RESULTADO: 65 comprobaciones correctas.

ANEXO: docs/trazas/memoria.txt

```
?SIMULADOR KERNEL | RAM 100 MB | Round Robin q=3 | escriba ayuda
```

kernel> crear A 60 5

Creado PID=1

kernel> crear Exceso 41 2

ERROR: Out of Memory: se necesitan 41 MB contiguos; libres=40 MB. Puede existi

r fragmentación externa.

kernel> estado

Reloj=0 q=3 despachos=0 cambios entre PID=0

RAM: 60/100 MB; libres=40 MB; bloques=[Bloque[pid=1, inicio=0, tamanio=60]]

Listos: [1]

Semáforo=1 propietario=null espera=[]

Recursos (recurso=PID): {}; solicitudes (PID=recurso): {}

```
Variable global=0
Deadlock: []
  1 A             LISTO             RAM= 60 MB CPU=5/5 espera=0 bloqueo=0
 respuesta=-1 fin=-1

kernel> abortar 1
Proceso abortado; memoria y recursos liberados.
Sin deadlock de recursos.
kernel> crear Completo 100 1
Creado PID=2
kernel> ejecutar
CPU [0,1) PID=2 restante=0
kernel> estado
Reloj=1 q=3 despachos=1 cambios entre PID=0
RAM: 0/100 MB; libres=100 MB; bloques=[]
Listos: []
Semáforo=1 propietario=null espera=[]
Recursos (recurso=PID): {}; solicitudes (PID=recurso): {}
Variable global=0
Deadlock: []
  1 A             ABORTADO          RAM= 60 MB CPU=5/5 espera=0 bloqueo=0
 respuesta=-1 fin=0
  2 Completo      TERMINADO         RAM=100 MB CPU=0/1 espera=0 bloqueo=0
 respuesta=0 fin=1

kernel> salir
```

ANEXO: docs/trazas/round-robin.txt

```
?SIMULADOR KERNEL | RAM 100 MB | Round Robin q=3 | escriba ayuda
```

kernel> crear A 10 5

Creado PID=1

kernel> crear B 10 4

Creado PID=2

kernel> crear C 10 2

Creado PID=3

kernel> ejecutar

CPU [0,3) PID=1 restante=2

CPU [3,6) PID=2 restante=1

CPU [6,8) PID=3 restante=0

CPU [8,10) PID=1 restante=0

CPU [10,11) PID=2 restante=0

kernel> estado

Reloj=11 q=3 despachos=5 cambios entre PID=4

RAM: 0/100 MB; libres=100 MB; bloques=[]

Listos: []

Semáforo=1 propietario=null espera=[]

Recursos (recurso=PID): {}; solicitudes (PID=recurso): {}

Variable global=0

Deadlock: []

1 A TERMINADO RAM= 10 MB CPU=0/5 espera=5 bloqueo=0 respuesta=0 fin=10

2 B TERMINADO RAM= 10 MB CPU=0/4 espera=7 bloqueo=0 respuesta=3 fin=11

3 C TERMINADO RAM= 10 MB CPU=0/2 espera=6 bloqueo=0

respuesta=6 fin=8

kernel> bitacora

t=0 CREAR PID=1 nombre=A RAM=10 CPU=5

t=0 CREAR PID=2 nombre=B RAM=10 CPU=4

t=0 CREAR PID=3 nombre=C RAM=10 CPU=2

t=3 CPU [0,3) PID=1 restante=2

t=3 FIN QUANTUM: guardar contexto PID=1 y enviar al final de listos

t=6 CPU [3,6) PID=2 restante=1

t=6 FIN QUANTUM: guardar contexto PID=2 y enviar al final de listos

t=8 CPU [6,8) PID=3 restante=0

t=8 TERMINADO PID=3: RAM y recursos liberados

t=10 CPU [8,10) PID=1 restante=0

t=10 TERMINADO PID=1: RAM y recursos liberados

t=11 CPU [10,11) PID=2 restante=0

t=11 TERMINADO PID=2: RAM y recursos liberados

kernel> salir

ANEXO: docs/trazas/semaforo.txt

```
?SIMULADOR KERNEL | RAM 100 MB | Round Robin q=3 | escriba ayuda
```

kernel> crear A 10 12

Creado PID=1

kernel> crear B 10 12

Creado PID=2

kernel> wait 1

Permiso concedido.

kernel> wait 2

Proceso bloqueado en semáforo.

kernel> sumar 2 1

ERROR: El proceso está bloqueado y no puede efectuar esa operación.

kernel> sumar 1 1

Variable global=1

kernel> signal 1

Permiso liberado/transferido.

kernel> sumar 2 1

Variable global=2

kernel> signal 2

Permiso liberado/transferido.

kernel> estado

Reloj=0 q=3 despachos=0 cambios entre PID=0

RAM: 20/100 MB; libres=80 MB; bloques=[Bloque[pid=1, inicio=0, tamanio=10], Bl

oque[pid=2, inicio=10, tamanio=10]]

Listos: [1, 2]

Semáforo=1 propietario=null espera=[]

Recursos (recurso=PID): {}; solicitudes (PID=recurso): {}

Variable global=2

Deadlock: []

1 A LISTO RAM= 10 MB CPU=12/12 espera=0 bloqueo

=0 respuesta=-1 fin=-1

2 B LISTO RAM= 10 MB CPU=12/12 espera=0 bloqueo

=0 respuesta=-1 fin=-1

kernel> salir

ANEXO: docs/trazas/deadlock.txt

```
?SIMULADOR KERNEL | RAM 100 MB | Round Robin q=3 | escriba ayuda
```

kernel> crear A 40 5

Creado PID=1

kernel> crear B 40 5

Creado PID=2

kernel> deadlock 1 2

ALERTA DEADLOCK [1, 2, 1]; use abortar PID.

kernel> estado

Reloj=0 q=3 despachos=0 cambios entre PID=0

RAM: 80/100 MB; libres=20 MB; bloques=[Bloque[pid=1, inicio=0, tamanio=40], Bl

oque[pid=2, inicio=40, tamanio=40]]

Listos: []

Semáforo=1 propietario=null espera=[]

Recursos (recurso=PID): {1=1, 2=2}; solicitudes (PID=recurso): {1=2, 2=1}

Variable global=0

Deadlock: [1, 2, 1]

1 A BLOQUEADO_RECURSO RAM= 40 MB CPU=5/5 espera=0 bloqueo=0

respuesta=-1 fin=-1

2 B BLOQUEADO_RECURSO RAM= 40 MB CPU=5/5 espera=0 bloqueo=0

respuesta=-1 fin=-1

kernel> detectar

ALERTA DEADLOCK [1, 2, 1]; use abortar PID.

kernel> abortar 1

Proceso abortado; memoria y recursos liberados.

Sin deadlock de recursos.

```
kernel> detectar
Sin deadlock de recursos.
kernel> ejecutar
CPU [0,3) PID=2 restante=2
CPU [3,5) PID=2 restante=0
kernel> estado
Reloj=5 q=3 despachos=2 cambios entre PID=0
RAM: 0/100 MB; libres=100 MB; bloques=[]
Listos: []
Semáforo=1 propietario=null espera=[]
Recursos (recurso=PID): {}; solicitudes (PID=recurso): {}
Variable global=0
Deadlock: []
  1 A              ABORTADO            RAM= 40 MB CPU=5/5 espera=0 bloqueo=0
 respuesta=-1 fin=0
  2 B              TERMINADO           RAM= 40 MB CPU=0/5 espera=0 bloqueo=0
 respuesta=0 fin=5

kernel> bitacora
t=0 CREAR PID=1 nombre=A RAM=40 CPU=5
t=0 CREAR PID=2 nombre=B RAM=40 CPU=5
t=0 SOLICITAR PID=1 R1 concedido
t=0 SOLICITAR PID=2 R2 concedido
t=0 SOLICITAR PID=1 R2 BLOQUEADO
t=0 SOLICITAR PID=2 R1 BLOQUEADO
t=0 ALERTA DEADLOCK: [1, 2, 1]
t=0 DESPERTAR PID=2
```

t=0 ABORTADO PID=1: RAM y recursos liberados

t=3 CPU [0,3) PID=2 restante=2

t=3 FIN QUANTUM: guardar contexto PID=2 y enviar al final de listos

t=5 CPU [3,5) PID=2 restante=0

t=5 TERMINADO PID=2: RAM y recursos liberados

kernel> salir